

2022/5/2021 王培鸿



深圳大学
Shenzhen University

地址: 深圳市南山区 网址: www.szu.edu.cn
传真: 0755-26534462 总机: 0755-26536114 邮政编码: 518060

4.5解: a. 数值为 0; 地址 6 的数值为 XFED3.

b. 1). $\therefore$ 地址 0 的数值为 00011110 0100 0011, 对应补码数为 7547

$\therefore$ 地址 1 数值为 1111 0000 0010 0101, 对应补码数为 -4059

2) ASCII 码为 'hul'e',

4) 无符号整数数值分别为 7547 和 61477.

乙: $\therefore$ 地址 0 的内容为 0001 1110 0100 0011

$\therefore$ 指令为 ADD R7, R1, R3.

丁: $\therefore$ 地址 5 的内容为 0000 0000 0000 0110, 对应地址为 0110, 即为地址 6.

该地址 6 存放的数值为 -301.

4.7.解: $\therefore$ 有 60 种操作码和 32 个寄存器

$\therefore$ IMM 的位数为 $32 - 5 - 6 - 32 - 6 - 5 - 5 = 14$ (位)

所以表达范围是 $-2^{13} \sim 2^{13}-1$.

5.4.解: a. $\therefore$ 内存有 256 位置且 $2^8 = 256$
所以至少为 8 位

b: $\therefore$ 需 $20 \text{ CE} \therefore 16 \leq 20 \leq 32$
所以要留出 5 位.

c: 偏移值应为 $10 - 3 - 1 = 6$
所以应为 6.

5-6 解: a: 0101 0110 1010 00100

b: 0101 0110 1010 01100

c: 101001 0110 1010 11111

d: 不能, 要 AND 指令中立即数的范围为 $-32 \sim 31$, 不能表达 $\#2^5$ ($\#32$).

5-8解: 因为 ADD 指令不能直接实现两个寄存器相加

2022152021

理由原因: $2^5 = 32$ 即要 5 bit 表达一个寄存器

王培鸿

$\therefore$ LC-3 指令长为 16 bit. ~~故剩下 11 位~~

故表达目标寄存器后剩下 7 位 < 10 位

所以 ADD 指令只能实现一个寄存器与立即数相加
不能实现两个寄存器相加

5-9 解: C 对应的指令为 NOP 指令.

区别: ADD 指令的操作数为寄存器数或立即数, 而两条指令的操作数只为立即数;
ADD 指令的操作码与两条指令不同; ADD 指令的功能是将寄存器 R_1 与 $\#0$ 相加, 并将结果送给 R_1 , 而两条指令的功能是判断上一个被修改的寄存器的值与 0 的关系.

5-11 解: 不能; 因为 ADD 指令的立即数为 5 位补码, 表达范围为 $-16 \sim +15$ 且 $-20 < -16$.

5-12 解: 因为

因为 R_0 和 R_1 为无符号整数.

所以有 $R_0 \square 51 = R_1 \square 51 = 0$ 时, R_2 中内容可信 (ZF), R_2 无溢出;

当 $R_0 \square 51 = R_1 \square 51 = 1$ 时, R_2 溢出, R_2 中内容不可信。

5-13 解: a: 使用指令 ADD $R_3, R_2, \#0$

b: 指令为 NOT R_3, R_3 ,
ADD $R_3, R_3, \#1$
ADD R_1, R_2, R_3

c: 指令为 ADD $R_1, R_1, \#0$

d: 不能, 因为条件码 N, Z, P 三者存在同一时刻有且只有一个为 1.

~~假设条件码 N 为 1, Z~~

e: 指令为 AND $R_2, R_2, \#0$

5-14 解: (2) 1001 101 010 11111

(4) 1001 011 110 11111

5-16 解:

5-15

~~$R_1 = X3133$~~

~~$R_2 = X31341$~~

~~$R_3 = X31342$~~

~~$R_4 = X31343$~~

$R_1 = X3121$

$R_2 = X4566$

$R_3 = XABCD$ $R_4 = XABCD$

~~$R_4 = X4567$~~

2022/5/2021 王培涛



深圳大学

Shenzhen University

地址: 深圳市南山区 网址: www.szu.edu.cn

传真: 0755-26534462 总机: 0755-26536114 邮政编码: 518060

5-16解: a: LD; 因为 LD 指令可读取 PC 间隔 $\pm 2^0$ 的内容, 且充分利用 PC.

b: LDI; 因为 LDI 指令可通过间接寻址访问任意地址的数据, 且无 $\pm 2^0$ 偏移量的约束.

c: LDR; 因为可通过寄存器存放数组的首地址, 不断偏移增量即可访问数组元素; 且无需像 LDI 那样间接寻址, 从而提高效率.

5-22解: ① 数值为 X123B

② 能; 指令为 LDI R6, #63, 二进制形式为 1010 110 000 11111

5-23解: R2 的内容为 0001 0100 1000 0010.

5-25解: 程序如下所示.

0011 0000 0000 0000; 程序起始位置为 X3000

0010 010 0000 10000; 将 X3011 的数据传给 R2

0010 011-00 0010000; 将 X3012 的数据传给 R3

1001 100 011 11111;

0001 100 100 1 00001; $R_4 = -R_3$

0001 001 010 000 100; $R_1 = R_2 - R_3$

0000 001 000000 101; BRP +5 (若 $R_1 \geq 0$ 则 $R_1 = R_1 + 5$)

0000 100 000000 010; BRN +2 (若 $R_1 < 0$, 则 $R_1 = R_1 + 2$)

0101 001 001 1 00000; (若 $R_2 = R_3$, $R_1 = 0$).

0000 111 0000 00011; 无条件跳转 +

0001 001 011 1 00000; $R_1 = R_3$

0000 111 0000 00001; 无条件跳转 +

0001 001 010 1 00000; $R_1 = R_2$

1111 0000 0010 0101; 程序终止.

5-30解: 推出 $R_0 + R_1 = \text{H#XFFFF}$.

报告
代码
 $B = \frac{F}{IL}$ CB

5-32 解: $R_0 = R_0 + 1$.

5-33 解: R_5 的内容的二进制形式有且仅有 5 位为 1, 其余位均为 0.

5-40 解: 作用是作为 ~~是否~~ PC 是否跳转到 $PC + offset$ 地址的标志。
当 A 输出为 1, 则 PC 执行 PC 变为 $PC + offset$ 的操作;
反之, 不执行。